

Theory Questions :-

=====

- Q1. How do you identify a Dynamic Programming (DP) problem?
- Q2. What is memoization (top-down approach)?
- Q3. What is tabulation (bottom-up approach)?
- Q4. What is the difference between memoization and tabulation?
- Q5. Which is better: top-down or bottom-up? Why?
- Q6. How does Dynamic Programming improve time complexity?
- Q7. What is the trade-off between time and space in Dynamic Programming?
- Q8. What is space optimization in Dynamic Programming?
- Q9. Can all recursive problems be converted into Dynamic Programming?
- Q10. How do you convert a recursive solution into Dynamic Programming?
- Q11. What are the steps to solve a Dynamic Programming problem?
- Q12. What is a state in Dynamic Programming?
- Q13. What is a transition relation in Dynamic Programming?
- Q14. How do you define base cases in Dynamic Programming?
- Q15. What is a stack and how does it work?
- Q16. What are the basic operations of a stack?
- Q17. What is the LIFO principle?
- Q18. How is a stack implemented using an array and linked list?

MCQ Questions :-

=====

Q1. Which of the following conditions is most suitable for applying Dynamic Programming?

- A) Problem can be solved only using sorting
- B) Problem has overlapping subproblems and optimal substructure
- C) Problem requires only recursion
- D) Problem uses linked list only

Q2. What will be the time complexity of Fibonacci using memoization?

- A) $O(2^n)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$

Q3. Which data structure follows the LIFO principle?

- A) Queue
- B) Stack
- C) Array
- D) Tree

Q4. What will be the output of the following Java code?

```
Stack<Integer> st = new Stack<>();  
st.push(10);  
st.push(20);  
st.push(30);  
System.out.println(st.pop());  
A) 10  
B) 20  
C) 30  
D) Error
```

Q5. Which of the following is an example of a bottom-up Dynamic Programming approach?

- A) Recursive Fibonacci without cache
- B) Fibonacci using loop and array
- C) Binary Search
- D) DFS Traversal

Q6. What will happen if pop() is called on an empty stack?

- A) Last inserted value returned
- B) Null returned
- C) Stack Underflow / Exception
- D) Stack Overflow

Q7. What will be the output of the following Java code?

```
int dp[] = new int[5];  
dp[0] = 1;  
dp[1] = 1;  
  
for(int i=2; i<5; i++)  
{  
    dp[i] = dp[i-1] + dp[i-2];  
}  
  
System.out.println(dp[4]);  
A) 2  
B) 3  
C) 5  
D) 8
```

Q8. Which statement is correct about memoization and tabulation?

- A) Memoization always uses loops only
- B) Tabulation always uses recursion
- C) Memoization uses recursion with cache

D) Both are unrelated to DP

Q9. What will be the output of the following Java code?

```
Stack<String> st = new Stack<>();  
st.push("A");  
st.push("B");  
st.pop();  
st.push("C");  
System.out.println(st.peek());
```

- A) A
- B) B
- C) C
- D) Error

Q10. Which of the following problems is commonly solved using Dynamic Programming?

- A) Tower of Hanoi
- B) Fibonacci Series
- C) Bubble Sort
- D) Linear Search

Practice Questions :-

=====

Q1. Write a program to find the nth Fibonacci number using Dynamic Programming (tabulation method).

The Fibonacci series is: 0, 1, 1, 2, 3, 5, 8, ...

Input:

Enter value of n: 7

Output:

7th Fibonacci number is: 13

Q2. Write a program to find the minimum number of steps required to reduce a number n to 1.

Allowed operations:

- Subtract 1
- Divide by 2 (if divisible)
- Divide by 3 (if divisible)

Use Dynamic Programming.

Input:

Enter number: 10

Output:

Minimum steps required: 3

Explanation:

$10 \rightarrow 9 \rightarrow 3 \rightarrow 1$

Q3. Write a program to find the next greater element for each element in an array.

If no greater element exists, print -1.

Input:

Enter array elements: 4 5 2 10

Output:

$4 \rightarrow 5$

$5 \rightarrow 10$

$2 \rightarrow 10$

$10 \rightarrow -1$

Q4. Write a program to find the maximum possible sum of elements from an integer array such that no two selected elements are adjacent.

Input:

Enter size: 6

Enter elements: 5 10 15 20 2 30

Output:

Maximum sum is: 60

Explanation:

Selected elements: $10 + 20 + 30 = 60$

Q5. Write a program to count total number of ways to reach last index of array if each element represents maximum jump length.

Input:

Enter size: 5

Enter elements: 2 3 1 1 4

Output:

Total ways: 4